

LNCP

Local Network Chat Protocol

Protocol Specification Document — Version 1.0

Author	Juli Chaphidze
Submission	June 15, 2026
Version	1.0 (Final)

Table of Contents

1	Overview and Design Goals	2
	Design Goals	2
2	Terminology and Roles	3
3	Transport and Encoding Rules	4
4	Message Framing	5
4.1	UDP (Discovery)	5
4.2	TCP (Conversation)	5
5	Common Message Envelope	6
6	Message Types	7
6.1	DISCOVER — Discovery Broadcast (UDP, Initiator → broadcast)	7
6.2	HELLO — Handshake (TCP, Recipient → Initiator)	7
6.3	ACCEPT — Handshake Success (TCP, Initiator → Recipient)	7
6.4	REJECT — Handshake Failure (TCP, Initiator → Recipient)	8
6.5	TEXT — Chat Message (TCP, either direction)	8
6.6	ACK — Acknowledgement (TCP, either direction)	8
6.7	BYE — Connection Close Request (TCP, either direction)	8
7	Communication Workflow	9
	Step-by-Step	9
8	State Machines	10
8.1	Initiator	10
8.2	Recipient	10
8.3	Turn Discipline in CHATTING	10
9	Error Handling	11
9.1	Rejection Codes (carried in REJECT.code)	11
9.2	Error Conditions and Required Behaviour	11
10	Assumptions and Limitations	13
	Assumptions	13
	Limitations	13
11	Worked Example (Annotated Trace)	14

1 Overview and Design Goals

LNCP (Local Network Chat Protocol) is a custom application-layer protocol that lets two hosts on the same local area network (LAN) discover one another by nickname and hold a one-to-one text chat. It is loosely inspired by the request/response discipline of **HTTP** (typed messages, status-like accept/reject responses) and the **textual, line-oriented** style of **SMTP** (human-readable verbs such as `HELLO`, `BYE`).

Design Goals

Goal	How LNCP achieves it
Zero pre-configuration	Peers find each other by nickname using a UDP broadcast; no central server or DNS is needed.
Reliability where it matters	Discovery uses connectionless UDP (cheap, broadcastable); the conversation itself uses connection-oriented TCP .
Self-describing messages	Every message is a JSON object carrying an explicit <code>type</code> field and protocol version.
Unambiguous framing	TCP messages use a 4-byte length prefix so the receiver always knows where each message begins and ends.
Deterministic flow	The chat is strictly turn-based (simplex): each message is answered before the next is sent.
Robust failure handling	Deadlines, UUID validation, typed rejection codes, and clean <code>BYE</code> termination cover the common error cases.

2 Terminology and Roles

Term	Meaning
Initiator	The peer that starts communication. It broadcasts a discovery request and then listens on a TCP port.
Recipient	The peer that listens for discovery broadcasts. When it sees its own nickname, it connects to the Initiator over TCP.
Request UUID	A random version-4 UUID generated by the Initiator that uniquely identifies one discovery attempt. It ties together the broadcast, the handshake, and the chat session.
Deadline	An absolute point in time (Unix epoch seconds) after which the Initiator will no longer accept a connection/handshake for that request.
Frame	One length-prefixed message on the TCP stream.
Turn	One send-then-receive cycle. LNCP chat alternates turns starting with the Initiator.

Note on the word “simplex”

As required by the assignment, the data channel is operated in a half-duplex / simplex *discipline*: only one party “holds the floor” at a time, and a message must be answered before the next message may be sent. The underlying TCP socket is full-duplex, but LNCP rules forbid simultaneous transmission.

3 Transport and Encoding Rules

Aspect	Rule
Discovery transport	UDP, sent to the IPv4 broadcast address 255.255.255.255.
Well-known UDP port	54321 — all Recipients bind and listen here for DISCOVER.
Conversation transport	TCP. The Initiator listens on a port it advertises in the broadcast (default 55000).
Character encoding	All bytes on the wire are UTF-8 .
Payload format	Every message is a single JSON object .
Numbers	Timestamps and deadlines are JSON numbers (Unix epoch seconds, floating point). Ports and sequence numbers are JSON integers.
Max TCP message size	1 MiB (1,048,576 bytes). Frames declaring a larger size are rejected as a protocol error (denial-of-service guard).
Max UDP datagram	Recipients read up to 4096 bytes per datagram.

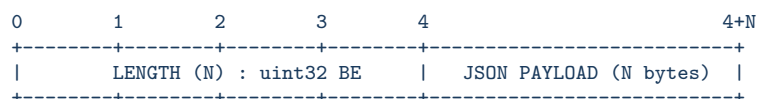
4 Message Framing

4.1 UDP (Discovery)

UDP is message-oriented, so **no framing** is applied. A DISCOVER message is the raw UTF-8 JSON object placed directly in one datagram.

4.2 TCP (Conversation)

TCP is a byte stream, so LNCP delimits each message with a **4-byte big-endian unsigned length prefix** followed by the JSON payload:



- LENGTH is the number of bytes in the JSON payload (not counting the 4 header bytes).
- The receiver reads exactly 4 bytes, decodes N, then reads exactly N more bytes. This guarantees correct message boundaries regardless of how TCP segments or coalesces the stream.
- LENGTH == 0 or LENGTH > 1 MiB is illegal and terminates the connection with a protocol error.

5 Common Message Envelope

Every LNCP message — UDP or TCP — shares the same four base fields:

Field	Type	Description
proto	string	Protocol identifier and version, always "LNCP/1.0".
type	string	The message type (see Section 6).
id	string	The request UUID this message belongs to.
ts	string	ISO-8601 UTC timestamp of when the message was created, e.g. "2026-06-14T03:43:50.934+00:00".

Type-specific fields are added on top of this envelope, as defined per message type below.

6 Message Types

LNCP defines seven message types. The four required by the assignment map as follows:

Assignment requirement	LNCP message type(s)
Handshaking message	HELLO
Handshaking response message	ACCEPT / REJECT
Text message	TEXT
Connection close request	BYE

The remaining types (DISCOVER, ACK) support discovery and the turn-based acknowledgement rule.

6.1 DISCOVER — Discovery Broadcast (UDP, Initiator → broadcast)

Announces that the Initiator wishes to chat with a named Recipient.

Field	Type	Description
proto	string	"LNCP/1.0"
type	string	"DISCOVER"
id	string	Request UUID
ts	string	Creation timestamp
recipient	string	Nickname of the intended chat partner
deadline	number	Absolute Unix-epoch time after which the request is dead
tcp_port	integer	TCP port the Initiator is now listening on

```
{
  "proto": "LNCP/1.0", "type": "DISCOVER",
  "id": "6bb7ca07-c193-447c-8988-bba8d8e4125d",
  "ts": "2026-06-14T03:43:50.934+00:00",
  "recipient": "tester", "deadline": 1781415845.9, "tcp_port": 55999
}
```

6.2 HELLO — Handshake (TCP, Recipient → Initiator)

Sent immediately after the Recipient connects. Proves the Recipient saw the broadcast by echoing the request UUID.

Field	Type	Description
<i>(envelope)</i>		type = "HELLO", id = the UUID from DISCOVER
sender	string	The Recipient's own nickname

6.3 ACCEPT — Handshake Success (TCP, Initiator → Recipient)

Field	Type	Description
<i>(envelope)</i>		type = "ACCEPT"
info	string	Human-readable welcome text

6.4 REJECT — Handshake Failure (TCP, Initiator → Recipient)

Field	Type	Description
<i>(envelope)</i>		type = "REJECT"
code	string	Machine-readable error code (see Section 9)
reason	string	Human-readable explanation

6.5 TEXT — Chat Message (TCP, either direction)

Field	Type	Description
<i>(envelope)</i>		type = "TEXT"
seq	integer	Per-sender, monotonically increasing sequence number (starts at 1)
body	string	The UTF-8 chat text

6.6 ACK — Acknowledgement (TCP, either direction)

Used to satisfy the “every message must be answered” rule when the receiver has no text to send back.

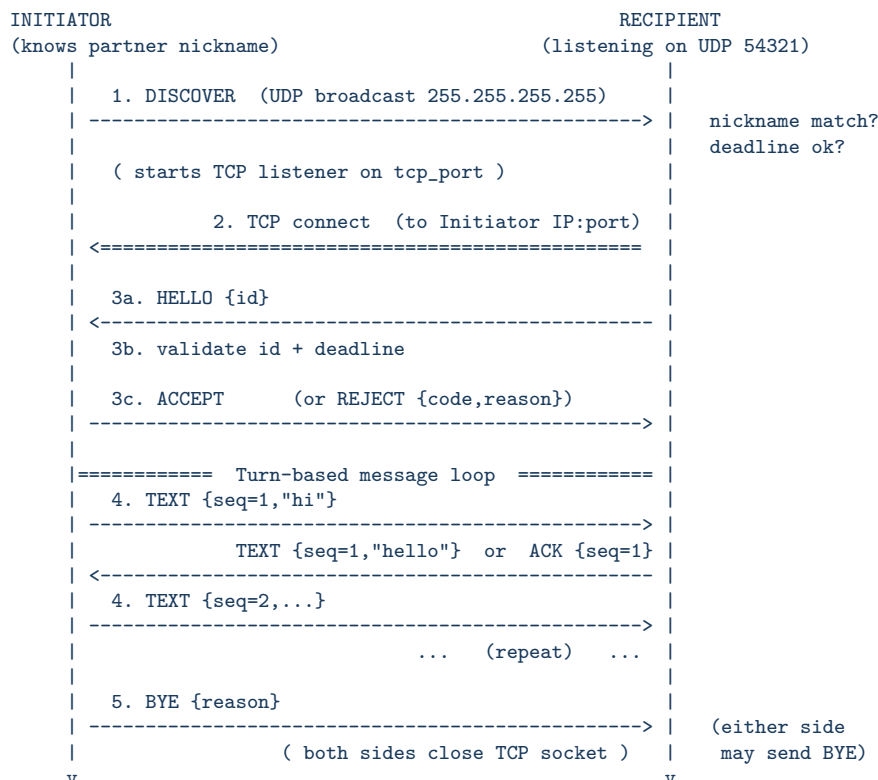
Field	Type	Description
<i>(envelope)</i>		type = "ACK"
seq	integer	The seq of the TEXT being acknowledged

6.7 BYE — Connection Close Request (TCP, either direction)

Field	Type	Description
<i>(envelope)</i>		type = "BYE"
reason	string	Why the session is being closed

After sending BYE, the sender closes the TCP socket. A peer that receives BYE stops the session and closes its socket too.

7 Communication Workflow



Step-by-Step

- Discovery (UDP).** The Initiator generates a random UUID, computes `deadline = now + N seconds`, binds nothing yet, and broadcasts a DISCOVER datagram on UDP 54321. It then opens a TCP listening socket on `tcp_port`.
- Connection (TCP).** Every Recipient on the LAN receives the datagram but only the one whose nickname equals `recipient` (and only if `now <= deadline`) reacts. It learns the Initiator's IP from the UDP packet's source address and opens a TCP connection to `IP:tcp_port`.
- Handshake.** The Recipient sends HELLO carrying the UUID. The Initiator verifies that the UUID matches its open request **and** that the deadline has not passed, then replies ACCEPT or REJECT.
- Message exchange.** On success the two sides exchange TEXT messages in strict alternation, the Initiator transmitting first each turn. Each received TEXT must be answered (with a reply TEXT or an ACK) before the other side transmits again.
- Termination.** Either side may send BYE at its turn. Both sides then close the TCP connection.

9 Error Handling

9.1 Rejection Codes (carried in REJECT.code)

Code	Cause	Raised by
UNKNOWN_UUID	HELLO.id does not match the Initiator's open request	Initiator
DEADLINE_EXPIRED	The handshake arrived after deadline	Initiator
VERSION_MISMATCH	Peer advertised an unsupported proto family	Initiator
PROTOCOL_ERROR	Out-of-sequence or structurally invalid message	Either

9.2 Error Conditions and Required Behaviour

#	Condition	Detected by	Action
1	No Recipient answers before the deadline	Initiator (accept() timeout)	Print notice, stop, exit cleanly.
2	Broadcast arrives after deadline	Recipient	Ignore the broadcast.
3	Broadcast for a different nickname	Recipient	Ignore the broadcast.
4	Malformed UDP datagram (bad JSON/UTF-8)	Recipient	Silently discard, keep listening.
5	TCP connect refused / times out	Recipient	Print error, abort attempt.
6	First TCP message is not HELLO	Initiator	Send REJECT(PROTOCOL_ERROR), close.
7	UUID mismatch in HELLO	Initiator	Send REJECT(UNKNOWN_UUID), close.
8	Deadline passed before HELLO validated	Initiator	Send REJECT(DEADLINE_EXPIRED), close.
9	Frame length 0 or > 1 MiB	Either	Raise protocol error, close connection.
10	Frame payload is not a valid JSON object	Either	Raise protocol error, close connection.
11	Peer closes socket unexpectedly	Either	Detected on read (empty recv) → report “connection lost”, close.

#	Condition	Detected by	Action
12	Ctrl-C / interrupt	Either	Best-effort BYE, then close socket.

Teardown Behaviour

All network sends during teardown are **best-effort**: if the socket is already broken, the error is swallowed so shutdown always completes.

10 Assumptions and Limitations

Assumptions

1. Both peers are on the same IPv4 LAN/subnet that forwards broadcast traffic.
2. The Recipient is already listening before the Initiator broadcasts (UDP broadcasts are not buffered or retransmitted in v1.0).
3. Nicknames are unique on the LAN; the first matching Recipient wins.
4. Host/OS firewalls permit UDP 54321 and the chosen TCP port.
5. Clocks are roughly synchronised enough for deadline comparison (deadlines are short, on the order of seconds).

Limitations (by design, for v1.0)

1. **No encryption or authentication.** Anyone on the LAN can read traffic or impersonate a nickname. LNCP is for trusted local networks only.
2. **One-to-one only.** An Initiator chats with exactly one Recipient per request; group chat is out of scope.
3. **Strict half-duplex.** A party cannot send two messages in a row; it must wait for the peer's reply. This is intentional per the assignment.
4. **IPv4 only.** Broadcast discovery does not cover IPv6 (which uses multicast).
5. **No message persistence or delivery receipts beyond ACK.**

11 Worked Example (Annotated Trace)

A real loopback session captured from the reference implementation:

INITIATOR -----	RECIPIENT -----
DISCOVER broadcast sent (id=6bb7ca07..., port=55999)	
	DISCOVER received (nickname matches)
	TCP connect -> 192.168.100.42:55999
Listening on TCP 55999 ...	
TCP connection accepted	
	HELLO {id=6bb7ca07...}
(id matches, deadline ok)	
ACCEPT {info:"Welcome!"}	
	Handshake ACCEPTED
-- session open --	
TEXT {seq:1, body:"hello from initiator"}	
	Initiator -> "hello from initiator"
	TEXT {seq:1, body:"hi back"}
Recipient -> "hi back"	
BYE {reason:"Closed by initiator."}	
	Initiator closed the session
(socket closed)	(socket closed)

This trace exercises **discovery, connection, handshaking, bidirectional messaging, and clean termination** — the full LNCP lifecycle.

End of LNCP/1.0 Specification.